SC2006 Software Engineering
Lecture 15 Supplement

Conrad Watt

Lt14 brief recap

- Factory pattern
- Façade pattern
- Observer pattern (again)

How does good software get written?

	Individual	Team	Organisation
Technical (*hard skills*)	coding style	architecture / infrastructure	infrastructure
Social (*soft skills*)	mentoring / knowledge transfer	dev methodology / chain-of-command	recruitment / management

this presentation is ***NOT*** exhaustive!!!
(and also not examinable)

How does good software get written - technical?

- Individual
  - write good code and pick good abstractions!

How does good software get written - technical?

- Individual
  - write good code!

- Learn the language/framework/dev team's ***idioms***
  - "idiom (idiomatic)": the preferred way of doing something that you can expect people to be familiar with

Individuals - technical - idioms

- Individual
  - write good code!

- Learn the language/framework/dev team's ***idioms***
  - indentation/bracketing style
    - my personal recommendation (if you have choice): "one-true-brace" style with 4 spaces for indentation

```
function {
    if (...) {
        doThing();
    } else {
        doOtherThing();
    } …
```

Individuals - technical - idioms

- Individual
  - write good code!

- Learn the language/framework/dev team's ***idioms***
  - indentation/bracketing style

  - most importantly, **be consistent with existing code**
  - some orgs use linters/formatters to enforce style rules

Individuals - technical - idioms

- Individual
  - write good code!

- Learn the language/framework/dev team's ***idioms***
  - indentation/bracketing style
  - name format - camelCase or snake_case ?

Individuals - technical - idioms

- Individual
  - write good code!

- Learn the language/framework/dev team's ***idioms***
  - indentation/bracketing style
  - name format - camelCase or snake_case ?
  - class, function, variable naming conventions e.g. :
    - names of factories should end with `Factory`
    - conversion functions should be named `X_to_Y`
    - string variable names should start with `str`
      (these are just possible examples!)

Individuals - technical - idioms

- Individual
  - write good code!

- Learn the language/framework/dev team's ***idioms***
  - indentation/bracketing style
  - names - camelCase or snake_case ?
  - class, function, variable naming conventions
  - language-specific idioms

Individuals - technical - idioms

For example if I search for “Java idioms” I get advice like:

when comparing `“StringLiteral”` to `stringVar`, write

```
“StringLiteral”.equals(stringVar)
```

rather than

```
stringVar.equals(“StringLiteral”)
```

to avoid Null Pointer Exceptions

Individuals - technical - idioms

when iterating over a List, often(*) prefer using Iterator or forEach

```
Iterator itr = list.iterator();
while(itr.hasNext()){
    MyObject name = itr.next();
}
```

rather than

```
for(int i =0; i<list.size; i++){
    MyObject name = list.get(i);
}
```

Individuals - technical - idioms

when iterating over a List, often(*) prefer using Iterator or forEach

```
list.forEach(elem -> {
    MyObject name = elem.get(i);
});
```

rather than

```
for(int i =0; i<list.size; i++){
    MyObject name = list.get(i);
}
```

Individuals - technical - idioms

reverse a String s by writing

```
new StringBuilder(s).reverse().toString();
```

rather than reading characters individually

How does good software get written - technical?

- Team
  - an organisation might be responsible for multiple software projects
  - generally smaller development teams will each take ownership of some project
  - technical decisions might need to be made at the org level across all projects, or delegated to the team

Teams - technical

- Team
  - determine a sensible system architecture

Teams - technical

- Team
  - determine a sensible system architecture
  - ensure work is divided well among team members
    - balance workload
    - don’t clobber each others’ work

Teams - technical

- Team
  - determine a sensible system architecture
  - ensure work is divided well among team members
    - balance workload
    - don’t clobber each others’ work
  - pick development languages/technologies/libraries

Teams - technical

- Team
  - determine a sensible system architecture
  - ensure work is divided well among team members
    - balance workload
    - don’t clobber each others’ work
  - pick development languages/technologies/libraries
  - pick style guide, possibly with linters/formatters

Teams - technical

- Team
  - determine a sensible system architecture
  - ensure work is divided well among team members
    - balance workload
    - don’t clobber each others’ work
  - pick development languages/technologies/libraries
  - pick style guide, possibly with linters/formatters
  - policies for deployment

Teams - technical

- Team
  - determine a sensible system architecture

  - ensure work is divided well among team members
    - balance workload
    - don’t clobber each others’ work

  - pick development languages/technologies/libraries
  - pick style guide, possibly with linters/formatters
  - policies for deployment
  - policies for committing/testing
    (e.g. code reviews / security audits / test coverage / fuzzing)

Teams - technical

- Team
  - determine a sensible system architecture
  - ensure work is divided well among team members
    - balance workload
    - don’t clobber each others’ work
  - pick development languages/technologies/libraries
  - pick style guide, possibly with linters/formatters
  - policies for deployment
  - policies for committing/testing
    (e.g. code reviews / security audits / test coverage / fuzzing)
  - policies for documentation

Teams - technical

- Team
  - determine a sensible system architecture
  - ensure work is divided well among team members
    - balance workload
    - don’t clobber each others’ work
  - pick development languages/technologies/libraries
  - pick style guide, possibly with linters/formatters
  - policies for deployment
  - policies for committing/testing
    (e.g. code reviews / security audits / test coverage / fuzzing)
  - policies for documentation
    - some of the above choices may be decided at a higher level in the organisation

How does good software get written - technical?

- Organisation
  - Ensure sufficient tools/equipment for teams
    - development environments (computer and IDE)
    - office space and meeting rooms
    - wider infrastructure (servers, cloud capacity)

How does good software get written - technical?

- Organisation
  - Ensure sufficient tools/equipment for teams
    - development environments (computer and IDE)
    - office space and meeting rooms
    - wider infrastructure (servers, cloud capacity)

  - Standardise some practices between teams - e.g.
    - language/library choice
    - cloud service provider
    - deployment and testing policies

How does good software get written - technical?

- Organisation
  - Ensure sufficient tools/equipment for teams
    - development environments (computer and IDE)
    - office space and meeting rooms
    - wider infrastructure (servers, cloud capacity)

  - Standardise some practices between teams - e.g.
    - language/library choice
    - cloud service provider
    - deployment and testing policies

  - Manage contractual/financial obligations

How does good software get written - social?

- Individual
  - Understand what other team members are doing
  - Other team members might also need to know what you are doing!

How does good software get written - social?

- Individual
  - Understand what other team members are doing
  - Other team members might also need to know what you are doing!
  - Learn from more experienced team members
  - Pass on your knowledge to less-experienced team members

How does good software get written - social?

- Team
  - Set the development methodology e.g.
    - agile or plan-driven development?
    - weekly stand-up meetings?
    - Kanban board?
    - performance evaluations?

How does good software get written - social?

- Team
  - Set the development methodology e.g.
    - agile or plan-driven development?
    - weekly stand-up meetings?
    - Kanban board?
    - performance evaluations?
  - Ensure responsibilities are understood among team members

How does good software get written - social?

- Team
  - Set the development methodology e.g.
    - agile or plan-driven development?
    - weekly stand-up meetings?
    - Kanban board?
    - performance evaluations?
  - Ensure responsibilities are understood among team members
  - Nominate/delegate to sub-team leads - decide long-term ***ownership*** of particular problems/sub-systems

How does good software get written - social?

- Team
  - Set the development methodology e.g.
    - agile or plan-driven development?
    - weekly stand-up meetings?
    - Kanban board?
    - performance evaluations?
  - Ensure responsibilities are understood among team members
  - Nominate/delegate to sub-team leads - decide long-term ***ownership*** of particular problems/sub-systems
  - Bid for organisation resources (e.g. headcount, equipment)

How does good software get written - social?

- Organisation
  - Advertising / PR / client relations

How does good software get written - social?

- Organisation
  - Advertising / PR / client relations
  - Servant leadership - understand team problems and use central power to solve them

How does good software get written - social?

- Organisation
  - Advertising / PR / client relations
  - Servant leadership - understand team problems and use central power to solve them
  - Develop company culture
    e.g. through paid retreats, office perks etc

How does good software get written - social?

- Organisation
  - Advertising / PR / client relations
  - Servant leadership - understand team problems and use central power to solve them
  - Develop company culture
    e.g. through paid retreats, office perks etc
  - Hire and assign programmers and other team members!